

ALGORITMO Y ESTRUCTURAS DE DATOS



TÍTULO DEL PROYECTO:
Métodos de ordenamiento de vectores

TRABAJO FINAL **INTEGRADOR**

UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL DE TUCUMÁN

INGENIERÍA EN SISTEMAS DE INFORMACIÓN
1K05

- Apás, Thiago
- Bustos, Gerónimo
- Novillo Daruich, Joaquin

ÍNDICE

1. Introducción

- ¿Qué son los métodos de ordenamiento?
- ¿Para que sirven?
- ¿Cómo se mide su eficiencia?
- Big O

2. Inserción

- ¿Qué hace?
- ¿Cómo funciona?
- ¿Cuál es su complejidad?

3. Quick Sort

- ¿Qué hace?
- ¿Cómo funciona?
- ¿Cuál es su complejidad?

4. Merge Sort

- ¿Qué hace?
- ¿Cómo funciona?
- ¿Cuál es su complejidad?

5. Radix Sort

- ¿Qué hace?
- ¿Cómo funciona?
- ¿Cuál es su complejidad?

6. Comparación

- Mejor caso y peor caso
- Complejidad
- Tiempo de ejecución

Metodos de Ordenamiento:

En este trabajo hicimos un algoritmo que carga un vector y aplica sobre el distintos métodos de ordenamiento, midiendo en cada caso el rendimiento en términos de comparaciones, intercambios y tiempo de ejecución. Podemos entender que el análisis y comparación son una fuerte herramienta para poder tener una critica a la hora de ordenar vectores o datos a futuro. Utilizamos estructuras secuenciales y recursivas según en este codigo.

Burbuja

Inserción

Quicksort

Countingsort

Mergesort

Radixsort

¿Qué son?

Son algoritmos diseñados para reorganizar los elementos de una estructura de datos, como un vector, según un criterio específico (por ej., de menor a mayor). El objetivo es lograr una secuencia útil para búsquedas, análisis, procesamiento o visualización.

¿Para qué sirven?

- Facilitan la búsqueda rápida de datos (como en búsquedas binarias).
- Permiten organizar información para tareas posteriores (como análisis estadísticos).
- Mejoran la eficiencia de algoritmos que dependen del orden, como en bases de datos o sistemas operativos.
- Ayudan a detectar patrones o duplicados en conjuntos de datos grandes.

¿Cómo se mide la eficiencia?

Se mide por su complejidad temporal, que representa cómo crece el tiempo de ejecución en función de la cantidad de datos de entrada (n), y por su complejidad espacial, que indica el uso de memoria.

"BIG O"

Es una forma de describir cómo crece el tiempo o el uso de memoria de un algoritmo a medida que aumenta la cantidad de datos que procesa. Es como una lupa matemática que te dice qué tan bien se comporta un algoritmo cuando el problema se hace grande.

¿Qué mide?

- Mejor caso: comportamiento óptimo del algoritmo.
- Peor caso: situación más desfavorable (la que más recursos consume).
- Caso promedio: comportamiento esperado en condiciones normales.

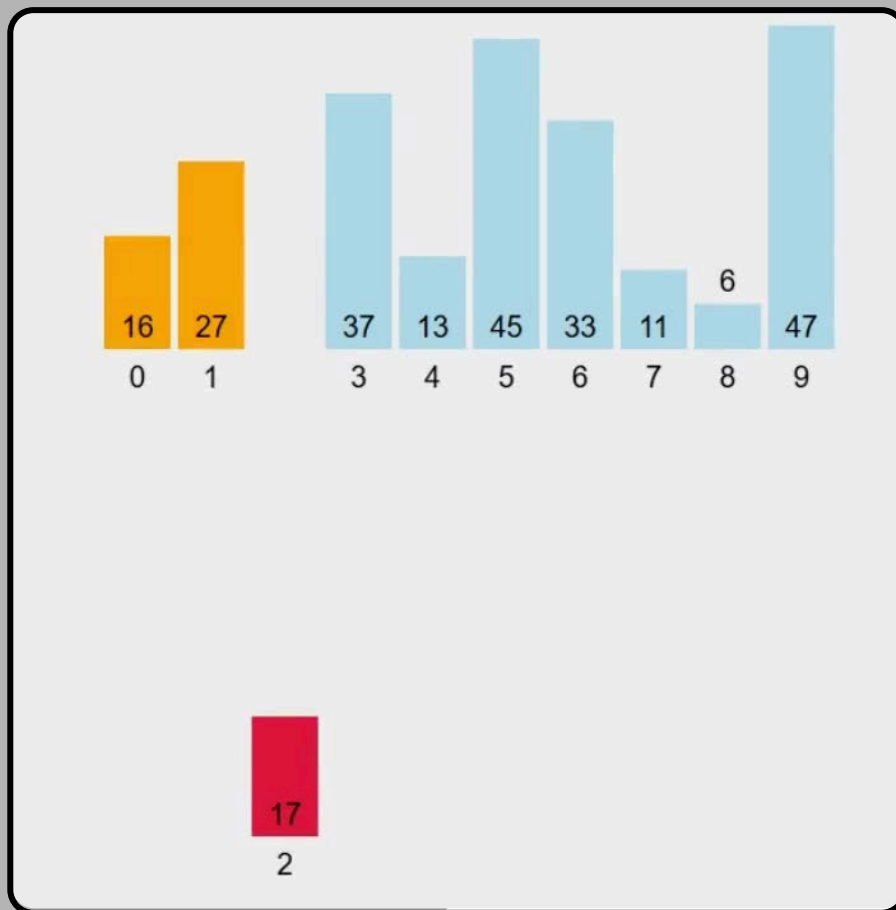
Notación	Nombre	Ejemplo	Crecimiento
$O(1)$	Tiempo constante	Acceder a un elemento	Muy rápido
$O(\log n)$	Logarítmico	Búsqueda binaria	Eficiente
$O(n)$	Lineal	Recorrer un vector completo	Moderado
$O(n \log n)$	Lineal-logarítmico	QuickSort, MergeSort	Bueno para ordenar
$O(n^2)$	Cuadrático	Doble bucle anidado	Lento con muchos datos
$O(2^n)$ y $O(n!)$	Exponencial	Algoritmos de fuerza bruta	Muy lento

1

Inserción

Funcionamiento:

- Recorre el vector desde el segundo elemento.
- Compara el elemento actual con los anteriores.
- Lo "inserta" en su posición correcta desplazando los mayores hacia la derecha.
- Es eficiente si el vector ya está parcialmente ordenado.



Complejidad:

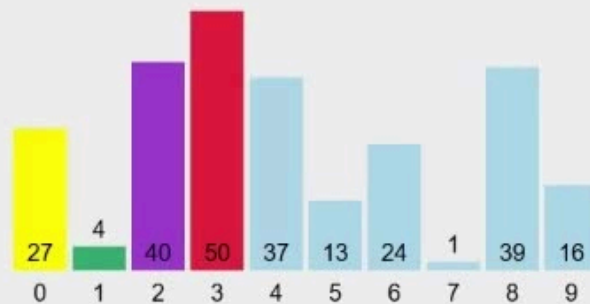
- Mejor caso (ordenado): $O(n)$
- Peor caso (inversamente ordenado): $O(n^2)$
- Promedio: $O(n^2)$
- Espacio adicional: $O(1)$

2

QuickSort

Funcionamiento:

1. **Elegir el pivote:** Normalmente se selecciona el último elemento del vector.
2. **Particionar el vector:** Se reorganiza el vector de modo que:
 - Todos los elementos menores o iguales al pivote (si el orden es ascendente) queden a su izquierda.
 - Todos los elementos mayores queden a su derecha.
3. **Aplicar recursividad:** Se repite el proceso en cada subvector (izquierda y derecha del pivote).
4. **Condición de parada:** La recursividad se detiene cuando los subvectores tienen cero o un elemento.



Complejidad:

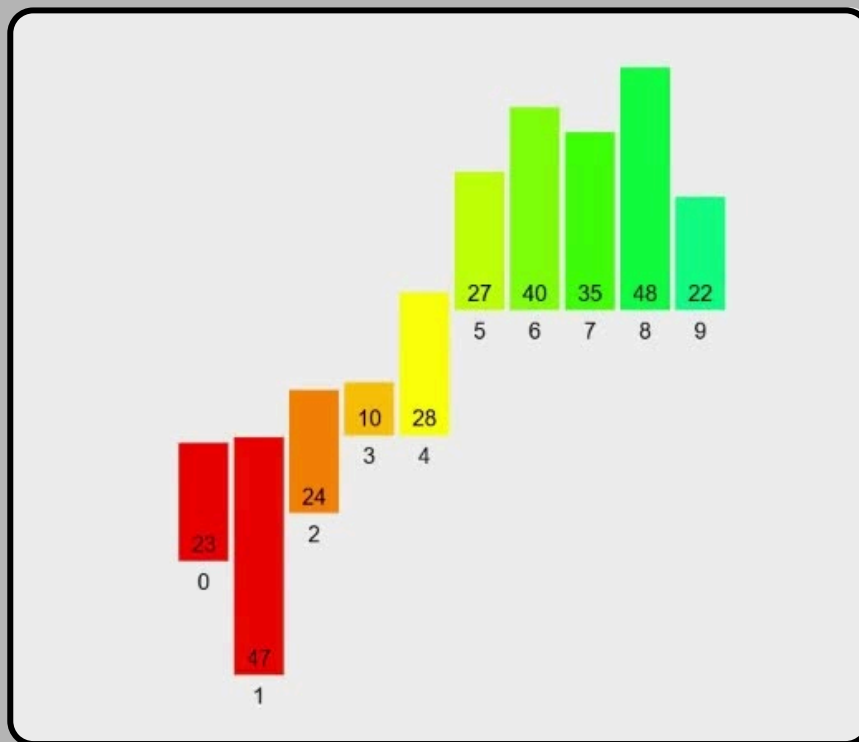
- Mejor caso: cuando el pivote divide el vector en partes iguales → eficiencia cercana a $n \log n$.
- Peor caso: cuando el pivote es el mínimo o máximo en cada partición → tiempo cuadrático n^2 .
- Promedio: $n \log n$, lo que lo vuelve muy competitivo frente a otros métodos.

3

MergeSort

Funcionamiento:

- 1.División: Se divide el vector en dos mitades.
- 2.Recursividad: Cada mitad se ordena aplicando nuevamente MergeSort.
- 3.Combinación: Se mezclan ambas mitades ordenadas en una nueva secuencia ordenada, utilizando una función auxiliar (merge).
- 4.Comparación: En la mezcla, se comparan los elementos de ambas mitades y se insertan de forma ordenada en el vector original.



Complejidad:

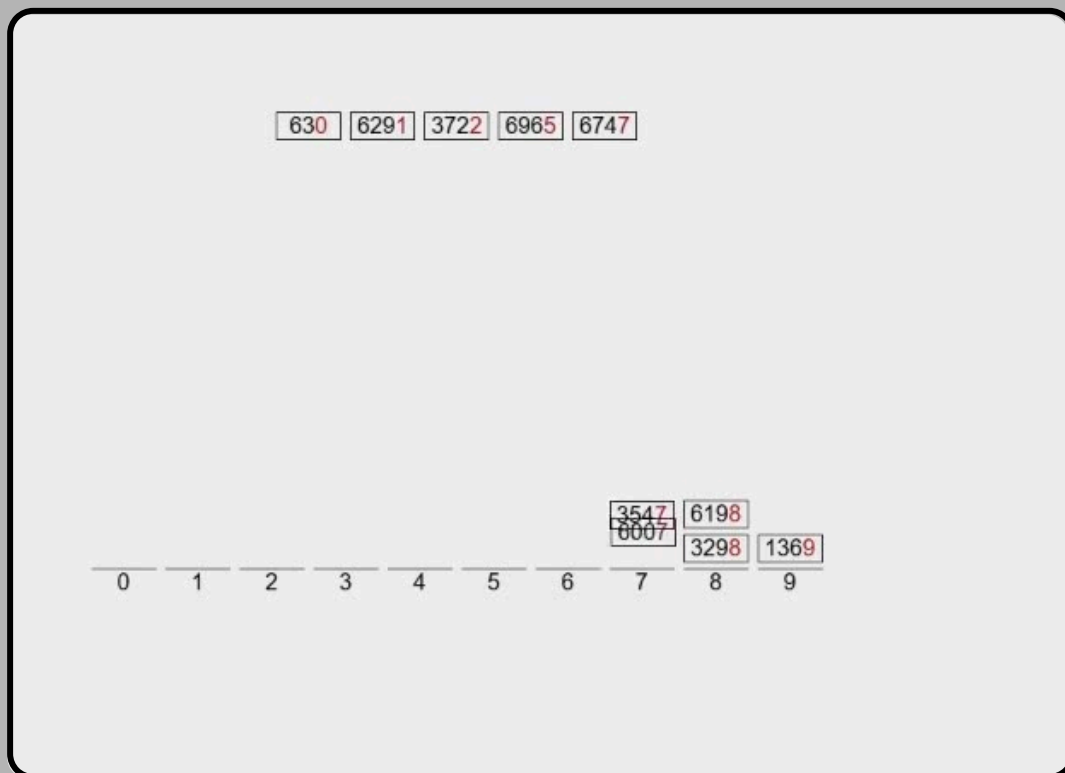
- Mejor caso: cuando las divisiones siempre equilibran bien las mitades → eficiencia $n \log n$.
- Peor caso: incluso en desorden total, mantiene divisiones equilibradas → también $n \log n$.
- Promedio: $n \log n$, lo que lo vuelve muy estable y predecible.

4

RadixSort

Funcionamiento:

1. Se determina la cantidad de dígitos del número más largo.
2. Se recorren los dígitos del menor al mayor (LSB a MSB).
3. En cada pasada, se agrupan los elementos según el dígito correspondiente.
4. Los buckets se concatenan en el nuevo vector ordenado.
5. Se repite el proceso por cada posición decimal.



Complejidad:

- **Mejor caso:** tiempo lineal proporcional a $n * k$, cuando los elementos tienen pocos dígitos y buena distribución.
- **Peor caso:** también $n * k$, si los elementos tienen muchos dígitos o generan buckets desbalanceados.
- **Caso promedio:** eficiencia de $n * k$, excelente si los datos son enteros de longitud acotada.

(n es la cantidad de elementos, k es la cantidad de dígitos en el número más largo.)

Código

1. Función Main()

- Declaramos el vector `vec[500000]` que se utilizará en todo el programa.
- Usamos un `do-while` que gestiona el menú.
- Según la opción elegida (`op`), llama a la función correspondiente.
- Para los métodos de ordenamiento, calculamos tiempo de ejecución con `clock()`.

2. Función Menu()

- Presenta las opciones disponibles al usuario.
- Valida que no se ingresen caracteres.
- Retorna la opción seleccionada para su procesamiento en `main()`.

3. Funciones de carga de datos:

Función cargaman(`vec`, `n`)

- Permite al usuario ingresar cada valor individualmente.
- Es útil para pruebas controladas con vectores pequeños.

Función cargaaut(`vec`, `n`)

- Solicita al usuario un tamaño, un mínimo y un máximo y el orden.
- Utiliza la función `rand()` para generar números pseudoaleatorios en el rango definido.
- Implementa verificación para evitar que el mínimo sea mayor al máximo.

4. Visualización del vector:

Función mostrar(vec, n)

- Imprime todos los valores del vector junto a su posición.
- Facilita la inspección del estado antes y después del ordenamiento.

Comparación

Cada algoritmo tiene ventajas las cuales son:

- **Inserción** es fácil de aplicar y funciona bien con vectores pequeños, pero su rendimiento empeora en vectores grandes debido a su complejidad cuadrática.
- **QuickSort** es eficiente en la mayoría de situaciones. Su estructura recursiva y su particionado inteligente permiten que sea rápido, pero su rendimiento puede empeorar si los pivotes no se eligen bien.
- **MergeSort** tiene una eficiencia constante sin importar el orden inicial de los datos. Es estable y predecible, aunque requiere más espacio por el uso de vectores auxiliares.
- **RadixSort** es bueno para ordenar grandes volúmenes de enteros con longitud acotada. No usa comparaciones directas, lo que lo hace rápido en contextos adecuados, pero menos versátil con datos más complejos o flotantes.

Algoritmo	Mejor Caso	Peor Caso	Espacio Extra	Estabilidad
Inserción	$O(n)$	$O(n^2)$	$O(1)$	Si
QuickSort	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
MergeSort	$O(n \log n)$	$O(n \log n)$	$O(n)$	Si
RadixSort	$O(nk)$	$O(NK)$	$O(n+k)$	Si